

PostgreSQL Performance Optimization

Techniques Used:

- 1. Indexing
- 2. Query Optimization
- 3. Data Partitioning

Indexing:

Indexes are data structures that allow PostgreSQL to locate rows without scanning the entire table. To speed up queries — especially those with:

- WHERE
- JOIN
- ORDER BY
- GROUP BY

Without an index: PostgreSQL scans every row.

With an index: PostgreSQL goes straight to the rows you care about.

How Does Indexing Work in PostgreSQL?

By default, PostgreSQL uses B-tree indexes (balanced tree).

B-tree Index:

- Data is organized like a binary tree (like a decision tree).
- At each step, PostgreSQL narrows down the possible values.
- Works great for: Numbers, Dates, Strings (text), Ranges

Example:

id	customer_id	status	order_date	total_amount
1	101	delivered	2024-01-01	300
2	102	pending	2024-01-05	500
...	...	...	...	...

For the above table named “orders”, we often run this query:

```
SELECT * FROM orders WHERE status = 'delivered';
```

without index, PostgreSQL checks every single row and this is slow for big tables.

But, with indexing:

```
CREATE INDEX idx_status ON orders(status);
```

PostgreSQL now looks at the index first, then jumps directly to matching rows, which is much faster!

Query Optimization:

Writing queries efficiently to reduce cost and improve execution time.

Example:

Avoiding SELECT *

Before:

```
SELECT * FROM users WHERE is_active = true;
```

➔ This fetches unnecessary columns (slow, high memory)

After:

```
SELECT id, name FROM users WHERE is_active = true;
```

➔ Fetches only required columns (faster, more efficient)

Data Partitioning:

Splitting a large table into smaller pieces (partitions) for better performance.

Example:

Before:

```
SELECT * FROM orders WHERE order_date >= '2024-01-01';
```

➔ PostgreSQL scans all rows.

After:

```
CREATE TABLE orders (  
    id SERIAL,  
    order_date DATE,  
    amount NUMERIC  
    ) PARTITION BY RANGE (order_date);  
  
CREATE TABLE orders_2024 PARTITION OF orders  
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

➔ Now, queries on 2024 data hit only that partition — massively faster.
